System Architecture

SC2006 Team 4 · Dialysis Transport Coordination System · Final Version

This document describes the high-level structure of the system: its components, the connectors through which they communicate, the architectural style that governs them, and the key design decisions on persistent data, access control, external integration and design patterns.

---

1. Architecture Diagram

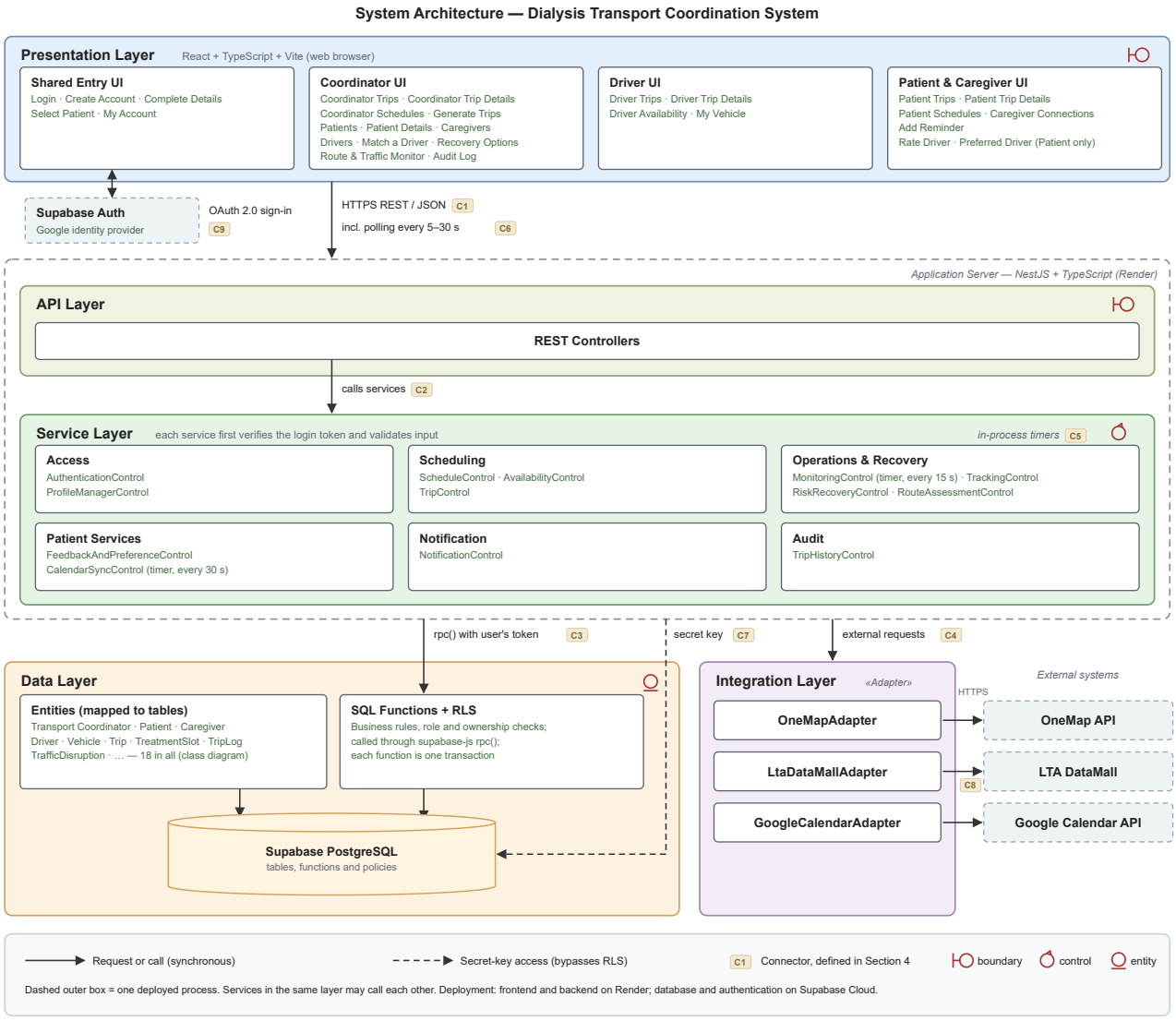


Figure 1. Layered client-server architecture. Connector labels C1–C9 are defined in Section 4. The Presentation Layer lists the main screens of each role; confirmation steps, the Schedule Form and exception states are shown in the Dialog Map.

2. Architectural Style

The system uses a **layered architecture** combined with a **client-server** style. The client is a React single-page application in the user's browser; the server is a NestJS application backed by a PostgreSQL

database that enforces most business and access rules itself. Open screens stay current by **polling** the API (connector C6), and background work is started by a timer inside the server (C5).

2.1 The style, stated as a style

An architectural style is a vocabulary of component and connector types, a topology and a set of constraints. For this system these are:

- **Component types:** boundary components (screens and REST controllers), control components (one service per area of the use case model, each starting with token and input validation; two of them also run on timers), entity components (the database tables, with the SQL functions and RLS policies that guard them) and adapter components (one per external service).
- **Connector types:** HTTPS request-reply (including periodic polling), local call-and-return, an in-process timer, supabase-js calls over HTTPS and an OAuth 2.0 sign-in redirect. These are set out in full in Section 4.
- **Topology:** four layers in a strict stack, with the Data Layer and the Integration Layer side by side beneath the Service Layer, as drawn in Figure 1.
- **Semantic constraints:** a layer may only call the layer directly below it; the browser never queries the database; every request except the sign-in redirect (C9) passes through the API Layer; the API Layer holds no business logic; services within the Service Layer may call each other; access rules for user data are enforced in the Data Layer; and no lower layer ever calls a layer above it, because the browser learns of changes by polling.

2.2 Design principles behind the style

- **Separation of concerns.** Presentation, task coordination, data with its rules, and external integration each sit in their own layer, so a change to one does not reach into the others.
- **Low coupling.** Layers depend on the interface of the layer below, not its implementation. Screens can change without touching scheduling rules, and apart from a few direct reads, services depend on named SQL functions rather than table layouts, so a table can change without touching the services as long as the function keeps its signature.
- **High cohesion.** Each service groups the logic for one area of the use case model, so the reason to change a service is a change to that area.
- **Modularity and abstraction.** SQL functions and adapters hide storage and external APIs behind interfaces owned by the application.
- **Single responsibility.** The API Layer receives and routes requests; the Service Layer authenticates and validates, then coordinates the steps of a task and calls external services; the Data Layer stores data and enforces its rules.
- **Traceability.** The layers match the boundary / control / entity stereotypes, so every class in the design class diagram belongs to exactly one layer. Section 3 maps each control class to the code module that implements it.

3. Components

Service-layer components keep the control-class names of the design class diagram. In the code they are grouped into fewer NestJS modules, so the mapping is many-to-one: the 13 control classes are implemented in five business modules (`auth`, `profiles`, `scheduling`, `drivers`, `trips`) plus `integrations`, and most operational controls live in `trips`. A separate `demo` module supports demonstrations only and is not part of the design model.

Every service starts the same way: it verifies the login token with Supabase Auth, rejects inactive accounts and roles that can never use the endpoint, and validates the request fields. Controllers pass the header and body through unchanged. Ownership and the detailed role rules are then enforced by the database (Section 5.2).

Component	Responsibility	Implemented in	Traces to
Shared Entry UI	Login; two-step signup (Create Account, Complete Details); Account Inactive; Select Patient for Caregivers linked to more than one Patient; My Account.	frontend `features/auth`, `features/profiles`	UC-01; Dialog Map — Shared Entry
Coordinator UI	Coordinator Trips and Coordinator Trip Details, Coordinator Schedules and Generate Trips, Patients and Patient Details, Caregivers, Drivers, driver matching, recovery options, route & traffic monitor, audit log.	frontend `features/*` pages	UC-02 to UC-05, UC-08; Dialog Map — Coordinator
Driver UI	Driver Trips and Driver Trip Details, Driver Availability, My Vehicle, Confirm Unavailability. While a trip is active, the driver's trip screen sends the live location in the background, so the Dialog Map needs no separate state for it.	frontend `features/trips`, `features/availability`, `features/drivers`	UC-06, UC-07, UC-09, UC-10; Dialog Map — Driver
Patient & Caregiver UI	Patient Trips and Patient Trip Details, which show live tracking within the page rather than on a separate screen; Patient Schedules; Caregiver Connections; confirm / cancel trip, rate and preferred driver (Patient only), add reminder.	frontend `features/trips`, `features/treatments`, `features/caregivers`, `features/calendar`	UC-02.1, UC-03, UC-09, UC-10, UC-11, UC-11.1, UC-12, UC-13; Dialog Map — Patient & Caregiver
REST Controllers	One endpoint per user action, e.g. `PATCH /trips/:id/cancel`.	`*.controller.ts` in each module	All use cases
Timers	Two control classes start their own in-process timers when the server starts. The traffic monitor (`MonitoringControl`) runs every 15 seconds. It claims assigned trips that are in progress, or whose pickup is between one day ago and one hour from now, assesses route and traffic, and flags trips at risk. It then claims trips that need recovery and generates recovery options for them automatically (UC-05.1). The calendar sync (`CalendarSyncControl`) runs every 30 seconds and publishes queued reminder changes to each connected user's Google Calendar.	`trips` (traffic monitor); `integrations` (calendar sync)	UC-05, UC-05.1, UC-08, UC-13; NFR-2.1, NFR-3.2

Component	Responsibility	Implemented in	Traces to
Access	`AuthenticationControl`, `ProfileManagerControl`: login, profile records, Caregiver-Patient links.	`auth`, `profiles`	UC-01, UC-02, UC-02.1
Scheduling	`ScheduleControl`, `AvailabilityControl`, `TripControl`: recurring trips, driver and vehicle assignment, availability, confirmation and cancellation.	`scheduling`, `drivers`, `trips`	UC-03, UC-03.1, UC-06, UC-07, UC-11, UC-11.1
Operations & Recovery	`MonitoringControl`, `TrackingControl`, `RiskRecoveryControl`, `RouteAssessmentControl`: trip status, live tracking, recovery options, route and ETA.	`trips`, using `integrations`	UC-04, UC-05, UC-05.1, UC-08, UC-09
Patient Services	`FeedbackAndPreferenceControl`, `CalendarSyncControl`: ratings, preferred driver, calendar reminders.	`trips`; `integrations` (calendar)	UC-12, UC-13
Notification	`NotificationControl`: records notifications for the affected users, which their browsers fetch by polling.	`trips`	UC-10
Audit	`TripHistoryControl`: records assignments, disruptions, recoveries, cancellations and final status; serves the Audit Log.	`trips`	UC-04.AF2; FR-6.4
Entities	The 18 entities of the design class diagram, mapped onto PostgreSQL tables. Most have their own table; some are part of another table, e.g. a Transport Coordinator is an account with the coordinator role. Rules about an entity are table constraints or checks inside the SQL functions that change it, e.g. a Trip can only be cancelled while upcoming.	`database/schema.sql`	Design class diagram
SQL Functions + RLS	Business operations run as SQL functions called through supabase-js `rpc()`; each runs as one transaction. Row-Level Security policies limit each user's reads to the rows they may see.	`database/schema.sql`	NFR-3.1, NFR-5.2, NFR-5.4
Supabase PostgreSQL	Relational database for all persistent data. The backend reaches it through supabase-js over HTTPS, normally with the user's token (C3) and for five specific uses with the secret key (C7).	Supabase Cloud	NFR-3.1
Supabase Auth	External identity service. The browser is redirected to it, and through it to Google, to sign in (C9); it verifies the Google identity and issues login tokens.	Supabase Cloud	UC-01; NFR-5.1
Integration adapters	OneMap, LTA DataMall and Google Calendar adapters over HTTPS. Each call times out after 5 seconds; bulk LTA fetches and route recalculation allow 10–20 seconds. OAuth 2.0 for Google Calendar.	`integrations`	UC-08, UC-13; NFR-2.1, NFR-7.1

4. Connectors

Shaw and Garlan define a software architecture as its components together with its **connectors** — the interactions among those components. The connectors below are labelled C1–C9 in Figure 1.

#	Between	Mechanism	Direction and timing
C1	Browser → API Layer	HTTPS request carrying JSON, with the login token in the authorisation header	Request-reply, synchronous. Every user action is one request.
C2	API Layer → Service Layer	Local method call inside the NestJS process	Call-and-return, synchronous.
C3	Service Layer → Data Layer (user)	supabase-js over HTTPS: `rpc()` to a SQL function, or a table read, carrying the user's login token	Request-reply, synchronous. RLS and the function's own checks apply as that user. Each function call is one transaction. The same client also verifies the login token with Supabase Auth.
C4	Service Layer → Integration Layer	Local method call on an adapter interface	Call-and-return, synchronous, with a timeout (5 seconds per call).
C5	Timer → Service Layer	In-process timers started by `MonitoringControl` (traffic monitor, every 15 seconds) and `CalendarSyncControl` (calendar sync, every 30 seconds) when the server starts	Time-triggered, asynchronous. No user request is involved.
C6	Browser → API Layer (polling)	Repeated HTTPS requests on the C1 path: trip progress every 5 seconds; notifications and driver availability every 15 seconds; the trip list and the signed-in account every 30 seconds	Request-reply, periodic. Keeps open screens current without a push channel.
C7	Service Layer → Data Layer (secret key)	supabase-js over HTTPS with the backend's secret key	Request-reply, synchronous. Bypasses RLS, so it is limited to five uses: the shared route cache, recovery-option generation, the traffic monitor, Google Calendar connections and reminder sync, and demo mode.
C8	Integration adapters → External APIs	HTTPS over the public internet; OAuth 2.0 bearer token for Google Calendar	Request-reply, synchronous, with a timeout (5 seconds per call; 10–20 seconds for bulk traffic data and route recalculation) and a defined failure path.
C9	Browser ↔ Supabase Auth and Google	OAuth 2.0 redirect over HTTPS	Request-reply, at sign-in only. Returns a login token, which the browser then sends on C1.

Why polling rather than push. Open screens fetch fresh data on a fixed interval (C6) instead of holding a push connection. This needs no separate WebSocket server, keeps every exchange a normal authorised request through the API Layer, and keeps the dependency direction downward: no lower layer ever calls the browser. **Why C7 is dashed.** It is the only connector that bypasses Row-Level Security, so its uses are kept to the five listed in Section 5.2.

5. Key Design Decisions

5.1 Identifying and storing persistent data

- **Persistent objects:** all 18 entity classes of the design class diagram — Transport Coordinator, Patient, Caregiver, CaregiverPatientLink, Driver, Vehicle, AvailabilityWindow, TreatmentSlot, DialysisCentre, Trip, TripLog, RouteAssessment, TrafficDisruption, RecoveryAction, Notification, DriverRating, DriverPreference and CalendarReminder.
- **Relational database:** PostgreSQL, because the data is highly connected (a trip links a patient, driver, vehicle, treatment slot and centre) and some changes must succeed together, e.g. cancelling a trip and releasing its driver.
- **Managed service:** Supabase hosts the database, so data survives application restarts and redeployments (NFR-3.1).
- **Rules in SQL functions:** business operations such as assigning a driver or cancelling a trip are SQL functions in the database, called through supabase-js `rpc()`. Services do not write SQL; a few simple reads use supabase-js table queries.
- **Entities mapped to tables:** most entities of the design class diagram have their own PostgreSQL table; some are part of another table, e.g. a Transport Coordinator is an account with the coordinator role. Rules about an entity are table constraints or checks inside the functions that change it, so they hold for every caller.
- **Atomic operations:** each SQL function runs as one transaction, so a use case that writes several records either fully succeeds or changes nothing. For example, cancelling a trip and releasing its driver and vehicle are committed together (UC-11.1).
- **Live location:** a driver's position is sent to the server as it is recorded, and viewers fetch it by polling. Only the latest position of an active trip is stored, and it is cleared when the trip ends (NFR-6.2).

5.2 Providing access control

- **Authentication:** users sign in with their Google account (OAuth 2.0) through Supabase Auth, which verifies the Google identity and issues a login token. There is no password form. This redirect (C9) is the only exchange that does not pass through the API Layer, and it returns only a token. Every later request must carry a valid token, which the server checks before doing anything else (NFR-5.1).
- **Role-based access control:** each account has exactly one role. Each service first rejects inactive accounts and roles that can never use the endpoint. The detailed role rules of the access matrix below are written in the database, as RLS policies and checks inside SQL functions (NFR-5.2, FR-1.3).
- **Ownership checks:** beyond the role, users only reach their own data. Patients see their own trips; Caregivers only Patients they are linked to (NFR-5.4); Drivers only trips assigned to them, with only the patient information needed for the trip (NFR-6.1). These checks are also enforced in the database.
- **Enforced in the database:** the backend reads and writes user data with the user's own token (C3), so PostgreSQL knows who is asking and applies RLS and the functions' checks. A missing check in one endpoint therefore does not by itself expose other users' rows. The backend's secret key bypasses RLS and is used for five purposes only (C7): the shared route cache; generating and storing recovery options; the background traffic monitor; Google Calendar connections and reminder sync, whose OAuth tokens are stored encrypted and never reach the browser; and demo mode. The browser never connects to the database for data.

Object / operation	Transport Coordinator	Driver	Patient	Caregiver
Patient records	view; update status and notes	—	edit own (My Account)	—
Caregiver-Patient links	view; unlink; re-activate with the Patient's authorisation	—	confirm a Caregiver's invitation; view; unlink	create an invitation; view; unlink
Treatment schedules	view; review change requests	—	create, edit, delete own	create, edit, delete linked Patients'
Trips	generate, assign, delete unassigned	—	—	—
Trip details	view all	view assigned	view own	view linked Patients'
Confirm / cancel upcoming trip	—	—	own	linked Patients'
Availability	view	update own; report unavailability	—	—
Recovery options	generate, approve	—	—	—
Driver live location	view active trips	send (own active trip)	view own active trip	view linked Patients' active trip
Driver rating, preferred driver	—	—	own	—
Calendar reminders	—	—	own	linked Patients'
Audit log	view	—	—	—

5.3 Integrating external services

- Each external service has its own adapter behind an interface owned by the application, so an API change affects one adapter only (NFR-7.1).
- Each routing and traffic call times out after 5 seconds; bulk LTA fetches and route recalculation allow 10–20 seconds. A timeout appears as the *Service Unavailable* state in the Dialog Map instead of stale data (NFR-2.1, UC-08.EX1).
- A Google Calendar failure only affects reminders; scheduling and monitoring continue (NFR-3.2, UC-13.EX1). Calendar access needs the user's OAuth consent (NFR-5.3, UC-13.EX2), and only the trip fields needed for the reminder are sent (NFR-6.3).

5.4 Design patterns

Pattern	Where it is used	Why
Layered architecture	Presentation, API, Service and Data Layers	Separation of concerns; each layer can be replaced without disturbing the others.
Client-server	Browser single-page application, NestJS server and database	Business rules and security are centralised on the server side, never in the browser.
Adapter	OneMap, LTA DataMall and Google Calendar adapters	An external API change affects one adapter only (NFR-7.1).
Database-enforced rules	RLS policies and SQL functions in PostgreSQL	Access and business rules are written once, next to the data, and apply to every request made with a user's token rather than being repeated in each endpoint.

6. Meeting the Non-Functional Requirements

The architecture exists to satisfy the quality attributes, not only the functions. The table below states which architectural mechanism delivers each one.

Quality attribute	Requirement	How the architecture delivers it
Security	NFR-5.1, NFR-5.2, NFR-5.4	Most data access is enforced by the database: requests run with the user's token, so RLS and the checks inside SQL functions apply, and a missing check in one endpoint does not by itself expose other users' rows. Secret-key access is limited to the five uses listed in Section 5.2, and the browser never connects to the database for data.
Reliability	NFR-3.1, NFR-3.2	Persistent state is held in a managed PostgreSQL instance rather than in the application process, so it survives restarts and redeployments. A Google Calendar failure is contained in one adapter and does not stop scheduling or monitoring.
Robustness	NFR-2.1	Each external call times out (5 seconds per call; 10–20 seconds for bulk traffic data and route recalculation) and surfaces as the *Service Unavailable* state in the Dialog Map, so the coordinator is shown a recoverable failure instead of stale or missing data.
Efficiency	NFR-2.1, NFR-6.2	Open screens poll on fixed intervals (C6): 5 seconds for trip progress, 15 seconds for notifications and availability, 30 seconds for the trip list and account. Background traffic checks only claim assigned trips that are in progress or whose pickup is between one day ago and one hour from now, so the load grows with active trips rather than with all records.
Maintainability	NFR-7.1	One adapter per external service, and each business operation in its own SQL function, so an external API change or a rule change is contained in one place.
Privacy	NFR-6.1, NFR-6.2, NFR-6.3	Each response is filtered by the viewer's role and link, by RLS in the database, before it reaches the browser. Only the latest position of an active trip is stored, and it is cleared when the trip ends.

7. Constraints

- **Languages:** TypeScript on both sides — React with Vite in the browser, NestJS on the server — so the team works in one language and toolchain. SQL (PL/pgSQL) in the database for business rules and access policies.
- **Libraries and services:** Tailwind CSS for styling; supabase-js for database and authentication access; Supabase for PostgreSQL and authentication; Playwright for browser tests and the Node.js test runner (via `tsx`) for backend tests. The ORM and test libraries named in the Lab 3 technology-stack exercise (Prisma, Jest, Vitest) were not adopted: database access goes through SQL functions and RLS (Sections 5.1, 5.2).
- **Communications:** HTTPS for all browser-to-server, server-to-database and server-to-external traffic, with periodic polling instead of server push; OAuth 2.0 for Google sign-in and Google Calendar. No other protocol is used.
- **Deployment:** frontend and backend on Render; database and authentication on Supabase Cloud. A single server process, with no microservices and no separate backend in another language — not justified by the current design at this scale.
- **Team:** the team already knows TypeScript, React, NestJS and Supabase, so the architecture deliberately avoids technologies that would need to be learned before the build starts.